ICT162

Proctored Online Exam - July Semester 2024

Object Oriented Programming

Thursday, 21 November 2024 **04:00 pm - 06:00 pm**

Time allowed: 2 hours

INSTRUCTIONS TO STUDENTS:

1. This examination contains **4** questions.
2. Answer all parts of each question in the corresponding answer box.
3. All answers must be typed in the answer box and submitted in Examena.
4. This is a **Closed-book** examination.
5. To prevent plagiarism and collusion, your submission will be reviewed by Turnitin. The Turnitin report will only be made available to the marker and you will not be able to view it.
6. The University takes plagiarism and collusion seriously, and your Turnitin report will be examined thoroughly as part of the marking process.
7. A4 Sized Cheat Sheet are allowed to be brought into exam venue

At the end of the examination.
To end the examination, please click on the "Submit and end exam" button to submit your answers.

THE UNIVERSITY RESERVES THE RIGHT NOT TO MARK YOUR SCRIPT IF YOU FAIL TO FOLLOW THESE INSTRUCTIONS.

ICT162 Copyright © 2024 Singapore University of Social Sciences (SUSS)

(Full marks: 100)

Question 1

Analyze the python classes below. Demonstrate your understanding of how class hierarchy and association are used in a typical object-oriented application.

```python
from abc import ABC, abstractmethod

class AAA(ABC):
    def __init__(self, x: int):
        self._varA = x

    @abstractmethod
    def method1(self, num: int):
        pass

class BBB(AAA):
    def __init__(self, x: int, y: int):
        super().__init__(x)
        self._varB = y

    def method1(self, num: int):
        return self._varB * num

class CCC(AAA):
    def __init__(self, x: int, y: int):
        super().__init__(x)
        self._varC = y

    @abstractmethod
    def method1(self, num: int):
         return self._varC + num

class DDD(BBB):
    def method2(self, num: int):
        return self.method1(num) * 2

class EEE(CCC):
    def method1(self, num: int):
        return super().method1(num * 2)
```

Carefully analyze each part (a) - (e) and assume that each part is independent.

- State whether there is an error in the Python statement(s).
- If there is an error, specify which statement causes that error and explain why the error occurs.
- If there is no error, explain what the statement(s) does (e.g. which class object is created) and state the output produced by the statements.

Question 1a

```python
b = BBB(2, 2)
print( b.method1(3) )
```

(4 marks)

Question 1b

```
c = CCC(3, 3)
print( c.method1(3) )
```

(4 marks)

Question 1c

```
d = DDD(3, 4)
print( d.method1(3) )
```

(4 marks)

Question 1d

```
d = DDD(3, 4)
print( d.method2(3) )
```

(4 marks)

Question 1e

```
e = EEE(4, 4)
print( e.method1(3) )
```

(4 marks)

Question 2

Always Good Services (AGS) is a car servicing provider. When a service job is requested, AGS will record the details of the job (job Id, estimated hours) and a service staff is assigned to carry out the request.

Study carefully the class diagram given in Figure Q2 which focuses on TWO types of job: ServiceJob and HomeServiceJob. Owners can request for home service job if they are not ready/available to send the vehicle to the workshop. The charges and job duration for any service job differ according to the assigned staff hourly rate, skill level, and the type of job.

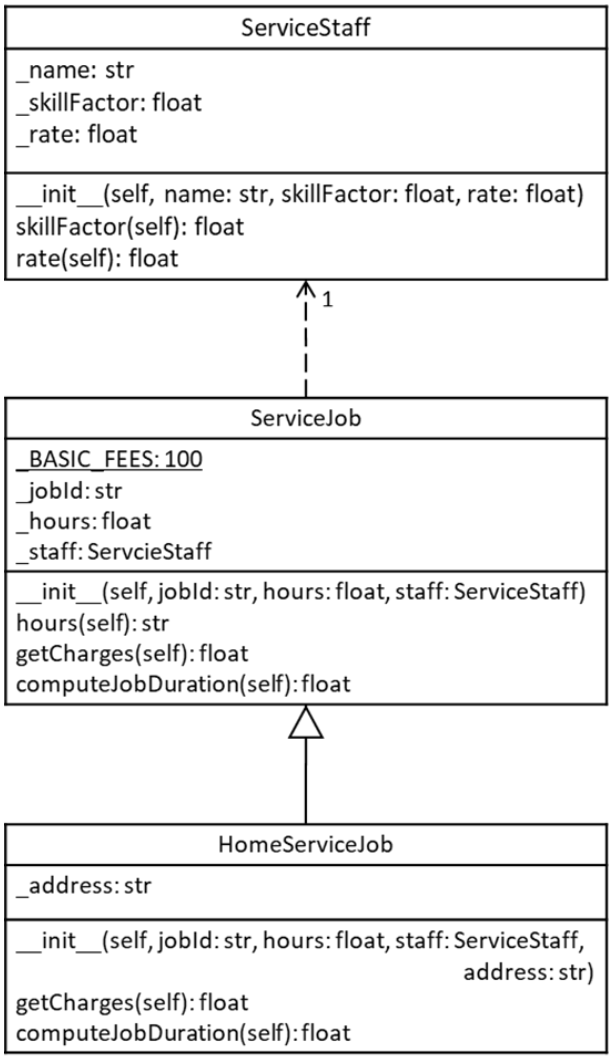


Figure Q2

Question 2a

Implement the class – ServiceStaff.

- It has THREE instance variables:
  - name (str): name of the service staff
  - skillFactor (float): skill level of the service staff. E.g., 1.2 or 1.15
  - rate (float): the hourly rate for the service staff. E.g., $55, $68.8
- Constructor initialises the three instance variables.
- Getter methods for skillFactor and rate. Use the property decorator.

(4 marks)

Question 2b

Implement the Python class definitions for ServiceJob.

- _BASIC_FEES: class variable that defines the basic fees for a service job.
  - It has THREE instance variables:
  - jobId (str): a unique job identifier like 'T123'
  - hours (float): the estimated hours to service the car

    - staff (ServiceStaff): the ServiceStaff object, representing the staff assigned to complete this service job
- Constructor initialises the three instance variables: _jobId, _hours and _staff.
- Getter method for hours. Use the property decorator.
- A computeJobDuration( ) method that computes and returns the job duration for this service job, which is the estimated hours divided by the assigned staff's skill factor. Hence, a highly skilled service staff is expected to complete the job faster.
- A getCharges( ) method that returns the charges for this service job, which is the sum of:
    - basic fees, as defined in the class variable; and
    - job duration multiply by staff's hourly rate

(9 marks)

Question 2c

Implement Python class definitions for HomeServiceJob.

- The HomeServiceJob class is a subclass of ServiceJob.
- Apart from jobId, hours and staff, the constructor initialises ONE additional instance variable: address (str), representing the location where the home service job is to be conducted.
- As the service job will be conducted away from the workshop, the assigned staff's skill factor will not help shorten the job duration. In fact, to manage customer expectation, the method computeJobDuration() will return the job duration as estimated hours plus 1 hour.
- Implement the method getCharges() such that a home service job will cost 1.5 times of standard service job.

(7 marks)

Question 2d

Based on your Python codes in Q2(c) for class HomeServiceJob, explain what type of overriding you have implemented for methods getCharges() and computeJobDuration().

(4 marks)

Question 2e

Referring to Figure Q2, for the following pairs of classes, identify between object composition and inheritance, and justify your choices:

- ServiceJob and HomeServiceJob
- ServiceJob and ServiceStaff
- HomeServiceJob and ServiceStaff

(6 marks)

Question 3

Study the class diagram in Figure Q3. You are to develop Python classes that raise exceptions.

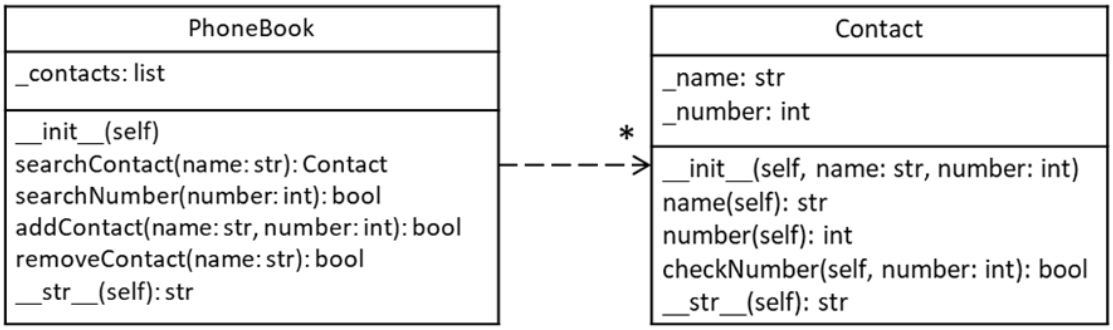


Figure Q3

Question 3a

Define the class InvalidContactException, which is a subclass of the Exception class. This class has no additional attribute or method.

(1 mark)

Question 3b

Implement the Contact class.

The Contact class has:

- TWO instance variables:
  - name (str): name of the contact
  - number (int): representing the 8-digit contact number
- Constructor that initialises the two instance variables. However, before the initialisations, the constructor must call the checkNumber method to validate the format of the contact number. The constructor does not handle the InvalidContactException exception object should it be raised by the checkNumber method
- Getter method for instance variables name and number. Use the property decorator.
- The checkNumber method validates that the contact number should be 8-digits and start with a 6, 8 or 9.
  - raise InvalidContactException with message: "Contact number must contain exactly 8 digits" if the length of the contact number is not 8
  - raise InvalidContactException with message: "Contact number must start with a 6, 8 or 9" if the contact number does not start with 6, 8 or 9
  - Otherwise, the method returns True
- The __str__ method returns a string representation of a Contact object. This is an example in the suggested format:
  Name: Joe Chen Number: 96884432

(11 marks)

Question 3c

Implement the class PhoneBook with the following specifications:

- An instance variable contacts which stores a collection (list) of Contact objects.
- A constructor without parameter for initialising the instance variable contacts to an empty list.
- A searchContact method that returns the Contact object whose name matches the parameter if such a contact exists. Otherwise, the method returns None.
- A searchNumber method that returns True if there is Contact object whose number matches the parameter. Otherwise, the method returns False.
- An addContact method that has name and number as parameters. The following task must be performed:
    - check if there is an existing contact with the same name (case insensitive). If yes, raise InvalidContactException with appropriate message
    - check if there is an existing contact with the same number. If yes, raise InvalidContactException with appropriate message
    - creates a Contact object with the given parameters and adds it to contacts. The method will not handle exception should the contact number does not match the required format.
    - The method returns True if the Contact object is added to contacts successfully
- The removeContact( ) method removes the Contact object whose name matches the given parameter and return True. Otherwise, the method returns False.
- The __str__ method returns a string representation of all the Contact objects, sorted in ascending order of the contact's name.

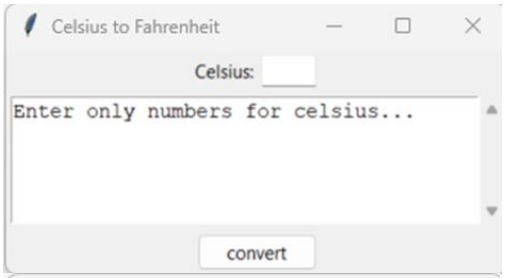
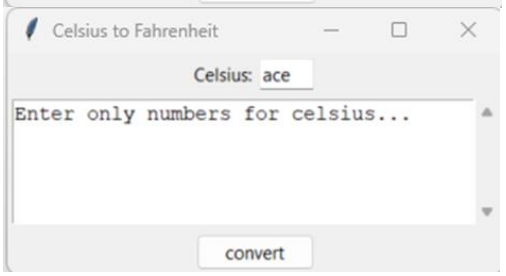
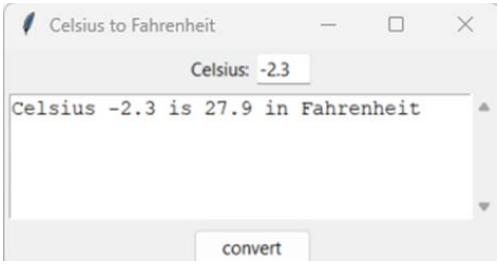
(18 marks)

Question 4

Figure Q4 below is a simple graphical user interface (GUI) in Python tkinter that does temperature conversion from Celsius to Fahrenheit.

Figure Q4

Some demonstrations of the GUI:

 	If the button is clicked when Celsius is empty or non-numeric, display a message.
	If Celsius is filled, perform the conversion by applying the formula: F = (C * 9/5) + 32 Display the Fahrenheit to 1 decimal place.

You are given the below partial program in Figure Q4(a):

```python
import tkinter as tk
from tkinter import ttk
from tkinter import scrolledtext

class GUI:

    def __init__(self):
        self._win = tk.Tk()
        self._win.resizable(True, True)
        self._win.title("Celsius to Fahrenheit")
        self.create_widgets()
        self._win.mainloop()

    def create_widgets(self):
        # creating of widgets
        celsius_lbl = ttk.Label(self._win, text="Celsius:")
        self._celsius_Ety = ttk.Entry(self._win, width=5)
        self._scrol_stxt = scrolledtext.ScrolledText(self._win,
                                                     width=38,
                                                     height=5,
                                                     wrap=tk.WORD)
        convert_btn = ttk.Button(self._win, text="convert",
                                 command=self.convert)

        # missing codes for laying out the widgets


    def convert(self):
        pass
```

Figure Q4(a)

Question 4a

Select a layout manager and provide the missing codes (Figure Q4a, line 25) to layout the widgets according to Figure Q4.

(8 marks)

Question 4b

Write statements that will implement the event handler for “convert” button, in other words, complete the callback function convert().

(12 marks)

----- END OF PAPER -----